

Lógica Computacional 2024/2025 - TP2

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: pip install pysmt
        pip install z3-solver
```

```
In [ ]: from pysmt.shortcuts import *
        from pysmt.typing import BVType
```

Exercício 2

Enunciado

Considere o problema descrito no documento Lógica Computacional: Multiplicação de Inteiros . Nesse documento usa-se um “Control Flow Automaton” como modelo do programa imperativo que calcula a multiplicação de inteiros positivos representados por vetores de bits.

Pretende-se

- a. Construir um SFOTS, usando BitVec's de tamanho n , que descreva o comportamento deste autómato; para isso identifique e codifique em z3 ou pySMT, as variáveis do modelo, o estado inicial, a relação de transição e o estado de erro.
- b. Usando k-indução verifique nesse SFOTS se a propriedade $(x * y + z = a * b)$ é um invariante do seu comportamento.
- c. Usando k-indução no FOTS acima e adicionando ao estado inicial a condição $(a < 2^{n/2}) \wedge (b < 2^{n/2})$, verifique a segurança do programa; nomeadamente prove que, com tal estado inicial, o estado de erro nunca é acessível.

```
assert a >= 0 and b >= 0
x , y, z = a , b , 0
0: while not y == 0:
1:   if even(y):
2:     x , y , z = x << 1 , y >> 1 , z
3:   else:
4:     x , y , z = x , y - 1, z + x
5: stop
# no final deve ser  z == a*b
```

a. Construir um SFOTS, usando BitVec's de tamanho n , que descreva o comportamento deste autómato; para isso identifique e codifique em z3 ou pySMT, as variáveis do modelo, o estado inicial, a relação de transição e o estado de erro.

Para modelar este programa como um SFOTS teremos o conjunto X de variáveis do estado dado pela lista `['x','y','z','pc']` .

Para definir as variáveis do modelo é definida a função `genState` que recebe a lista com o nome das variáveis do estado, uma etiqueta, um inteiro e o número de bits, e cria a i-ésima cópia das variáveis do estado para essa etiqueta. As variáveis lógicas começam sempre com o nome de base das variáveis dos estado, seguido do separador !.

```
In [150]- def genState(vars, s, i, nbits):
          state = {}
          for var in vars:
              state[var] = Symbol(f"{var}!{s}_{i}", BVType(nbits))
          return state
```

Para definir o estado inicial é definida a função `init` que, dado um possível estado do programa(um dicionário de variáveis), dois inteiros a e b, e o número de bits. Devolve um predicado do pySMT que testa se $x = a, y = b, z = 0, a \geq 0, b \geq 0$ são todos satisfazíveis, isto é, se esse estado é um possível estado inicial do programa.

```
In [151]- def init(state, a, b, nbits):

          x = Equals(state['x'], BV(a, nbits))
          y = Equals(state['y'], BV(b, nbits))
          z = Equals(state['z'], BV(0, nbits))
          pc = Equals(state['pc'], BV(0, nbits))
          apos = BVUGE(BV(a, nbits), BV(0, nbits))
          bpos = BVUGE(BV(b, nbits), BV(0, nbits))

          return And(x, y, z, pc, apos, bpos)
```

Para definir os estados de erro é definida a função `error` que, dado um estado do programa, devolve um predicado do pySMT que testa se esse estado é um possível estado de erro do programa.

Em Z3 as operações “shift” `<< 1` e a soma aritmética `z + x` são sempre totais: nunca assinalam um resultado de erro e dão sempre um resultado no domínio pretendido, mesmo que inesperado.

Numa ALU as operações são parciais e no “output”, para além dos eventuais resultados, podem activar vários bits de sinal; nomeadamente os bits de `overflow` e de `carry` .

Por isso uma situação onde o ALU ative o bit de `overflow` não pode ser simplesmente retirado do resultado da mesma operação em Z3. É preciso programar adequadamente a deteção desse evento.

Tendo isto em conta, definimos, em `error`, os casos em que um possível overflow ocorra. Isto é possível em quando em `pc = 2` quando $x = x << 1$ e em `pc = 4` quando $z = z + x$.

```
In [152]- def error(state, nbits):
          MAX_VAL = BV((1 << nbits) - 1, nbits) #para n=4 maxval seria 1111 em binário

          #pc = 2 overflow em left (x,y,z ~ 2*x, y/2, z)
          e2 = And(Equals(state['pc'], BV(2,nbits)), BVUGT(state['x'], BVUDiv(MAX_VAL, BV(2,nbits))))

          #pc = 4 overflow em right (x,y,z + x, y-1, z+x)
          e4 = And(Equals(state['pc'], BV(4,nbits)), BVUGT(BVAdd(state['z'], state['x']), MAX_VAL))

          return Or(e2, e4)
```

Para definir as relações de transição é definida a função `trans` que, dados dois estados do programa, devolve um predicado do pySMT que testa se é possível transitar do primeiro para o segundo estado.

Nos casos em que é necessária a verificação `if even(y)` é usada a função `even(x, nbits)` que vai verificar se o Least Significant Bit (LSB) do bitvec x é 0. Em binário, um número par (even) acaba sempre em 0 (o LSB é 0), já quando é ímpar (odd) acaba em 1 (o LSB é 1).

```
In [153]- def even(x, nbits):
          return Equals(BVAnd(x, BV(1,nbits)), BV(0,nbits))

def trans(curr, prox, nbits):
    t01 = And(Equals(curr['pc'],BV(0,nbits)), Not(Equals(curr['y'], BV(0,nbits))), Equals(prox['pc'],BV(1,nbits)), Equals(prox['x'],curr['x']), Equals(prox['y'],curr['y']), Equals(prox['z'],curr['z']))
    t12 = And(Equals(curr['pc'],BV(1,nbits)), even(curr['y'],nbits), Equals(prox['pc'],BV(2,nbits)), Equals(prox['x'],curr['x']), Equals(prox['y'],curr['y']), Equals(prox['z'],curr['z']))
    t20 = And(Equals(curr['pc'],BV(2,nbits)), Equals(prox['pc'],BV(0,nbits)), Equals(prox['x'], BVMul(curr['x'], BV(2, nbits))), Equals(prox['y'], BVUDiv(curr['y'], BV(2, nbits))), Equals(prox['z'],curr['z']))
    t14 = And(Equals(curr['pc'],BV(1,nbits)), Not(even(curr['y'],nbits)), Equals(prox['pc'],BV(4,nbits)), Equals(prox['x'],curr['x']), Equals(prox['y'],curr['y']), Equals(prox['z'], curr['z']))
    t40 = And(Equals(curr['pc'],BV(4,nbits)), Equals(prox['pc'],BV(0,nbits)), Equals(prox['x'],curr['x']), Equals(prox['y'],BVSub(curr['y'], BV(1, nbits))), Equals(prox['z'],BVAdd(curr['z'], curr['x'])))
    t05 = And(Equals(curr['pc'],BV(0,nbits)), Equals(curr['y'], BV(0,nbits)), Equals(prox['pc'],BV(5,nbits)), Equals(prox['x'],curr['x']), Equals(prox['y'],curr['y']), Equals(prox['z'],curr['z']))

    return Or(t01, t12, t14, t20, t40, t05)
```

Usamos `genTrace` para gerar um possível estado de execução com n transições.

```
In [154]- def genTrace(vars,init,trans,error,n,a,b,nbits):
          with Solver() as s:
              X = [genState(vars,'X',i,nbits) for i in range(n+1)] # cria n+1 estados (com etiqueta X)
              I = init(X[0],a,b,nbits)
              Tks = [ trans(X[i], X[i+1], nbits) for i in range(n) ]
              E = [error(X[i], nbits) for i in range(n+1)]

              if s.solve([I, And(Tks), Not(Or(E))]):
                  for i in range(n+1):
                      print(f"Estado: {i}")
                      for v in X[i]:
                          val = s.get_value(X[i][v])
                          print(f"          {v} = {val}")
                  return True
              else:
```

```
print("Overflow")
return False
```

Resultados

```
In [155]: vars = ['x', 'y', 'z', 'pc']
n = 10
nbits = 4
a = 4
b = 4

genTrace(vars, init, trans, error, n, a, b, nbits)
```

Overflow

```
Out[155]: False
```

```
In [156]: vars = ['x', 'y', 'z', 'pc']
n2 = 10
nbits = 4
a2 = 1
b2 = 3

genTrace(vars, init, trans, error, n2, a2, b2, nbits)
```

Estado: 0

```
  x = 1_4
  y = 3_4
  z = 0_4
  pc = 0_4
```

Estado: 1

```
  x = 1_4
  y = 3_4
  z = 0_4
  pc = 1_4
```

Estado: 2

```
  x = 1_4
  y = 3_4
  z = 0_4
  pc = 4_4
```

Estado: 3

```
  x = 1_4
  y = 2_4
  z = 1_4
  pc = 0_4
```

Estado: 4

```
  x = 1_4
  y = 2_4
  z = 1_4
  pc = 1_4
```

Estado: 5

```
  x = 1_4
  y = 2_4
  z = 1_4
  pc = 2_4
```

Estado: 6

```
  x = 2_4
  y = 1_4
  z = 1_4
  pc = 0_4
```

Estado: 7

```
  x = 2_4
  y = 1_4
  z = 1_4
  pc = 1_4
```

Estado: 8

```
  x = 2_4
  y = 1_4
  z = 1_4
  pc = 4_4
```

Estado: 9

```
  x = 2_4
  y = 0_4
  z = 3_4
  pc = 0_4
```

Estado: 10

```
  x = 2_4
  y = 0_4
  z = 3_4
  pc = 5_4
```

```
Out[156]: True
```

```
In [157]: vars = ['x', 'y', 'z', 'pc']
n3 = 10
nbits = 4
a3 = 5
b3 = 3

genTrace(vars, init, trans, error, n3, a3, b3, nbits)
```

```
Estado: 0
  x = 5_4
  y = 3_4
  z = 0_4
  pc = 0_4

Estado: 1
  x = 5_4
  y = 3_4
  z = 0_4
  pc = 1_4

Estado: 2
  x = 5_4
  y = 3_4
  z = 0_4
  pc = 4_4

Estado: 3
  x = 5_4
  y = 2_4
  z = 5_4
  pc = 0_4

Estado: 4
  x = 5_4
  y = 2_4
  z = 5_4
  pc = 1_4

Estado: 5
  x = 5_4
  y = 2_4
  z = 5_4
  pc = 2_4

Estado: 6
  x = 10_4
  y = 1_4
  z = 5_4
  pc = 0_4

Estado: 7
  x = 10_4
  y = 1_4
  z = 5_4
  pc = 1_4

Estado: 8
  x = 10_4
  y = 1_4
  z = 5_4
  pc = 4_4

Estado: 9
  x = 10_4
  y = 0_4
  z = 15_4
  pc = 0_4

Estado: 10
  x = 10_4
  y = 0_4
  z = 15_4
  pc = 5_4
```

Out[157]_ True

b. Usando k-indução verifique nesse SFOTS se a propriedade $(x * y + z = a * b)$ é um invariante do seu comportamento.

Sendo *inv* a função que define a propriedade a verificar se é invariante.

```
In [158]_ def inv(state, a, b, n):
          return Equals(BVAdd(BVMul(state['x'], state['y']), state['z']), BVMul(BV(a,n), BV(b,n)))
```

É usada a função *kinduction_always* que verifica se a propriedade *inv* é invariante por k-indução.

```
In [159]_ def kinduction_always(declare, init, trans, inv, k, a, b, nbits):
          with Solver(name="z3") as solver:
              s = [genState(vars, 'X', i, nbits) for i in range(k)]
              solver.add_assertion(init(s[0], a, b, nbits))
              for i in range(k-1):
                  solver.add_assertion(trans(s[i], s[i+1], nbits))

              for i in range(k):
                  solver.push()
                  solver.add_assertion(Not(inv(s[i], a, b, nbits)))
                  if solver.solve():
                      print(f"> Contradição! O invariante não se verifica nos k estados iniciais.")
                      for st in s:
                          print("x, pc, inv: ", solver.get_value(st['x']), solver.get_value(st['pc']))
                      return
                  solver.pop()

              s2 = [genState(vars, 'Y', i + k, nbits) for i in range(k + 1)]

              for i in range(k):
                  solver.add_assertion(inv(s2[i], a, b, nbits))
                  solver.add_assertion(trans(s2[i], s2[i+1], nbits))

              solver.add_assertion(Not(inv(s2[-1], a, b, nbits)))

              if solver.solve():
                  print(f"> Contradição! O passo indutivo não se verifica.")
                  for i, state in enumerate(s):
                      print(f"> State {i}: x = {solver.get_value(state['x'])}, pc = {solver.get_value(state['pc'])}.")
                  return
              print(f"> A propriedade verifica-se por k-indução (k={k}).")
```

Resultados

```
In [160]_ a_b=5
          b_b=3
          nbits=4
          k_b=10
          kinduction_always(genState, init, trans, inv, k_b, a_b, b_b, nbits)

> A propriedade verifica-se por k-indução (k=10).
```

c. Usando k-indução no FOTS acima e adicionando ao estado inicial a condição $(a < 2^{n/2}) \wedge (b < 2^{n/2})$, verifique a segurança do programa; nomeadamente prove que, com tal estado inicial, o estado de erro nunca é acessível.

Como vimos, podemos verificar propriedades de animação do tipo $F \neq$ usando BMC. Mais uma vez, se quisermos verificar estas propriedades para qualquer execução ilimitada temos que usar um procedimento alternativo. Uma possibilidade consiste em reduzir a verificação dessas propriedades à verificação de uma propriedade de segurança, mais concretamente um invariante, que possa ser verificado por indução.

```
In [161]_ def init_c(state, a, b, nbits):

          x = Equals(state['x'], BV(a, nbits))
          y = Equals(state['y'], BV(b, nbits))
          z = Equals(state['z'], BV(0, nbits))
          pc = Equals(state['pc'], BV(0, nbits))
          apos = BVUGE(BV(a, nbits), BV(0, nbits))
          bpos = BVUGE(BV(b, nbits), BV(0, nbits))
          cond = And(BVULT(BV(a, nbits), BV(2 ** (nbits//2), nbits)), BVULT(BV(b, nbits), BV(2 ** (nbits//2), nbits)))

          return And(x, y, z, pc, apos, bpos, cond)
```

Usando a k-indução definida anteriormente (*kinduction_always*)

```
In [162]_ a_c=5
          b_c=3
          nbits=4
          k=4
          kinduction_always(genState, init_c, trans, inv, k, a_c, b_c, nbits)

> A propriedade verifica-se por k-indução (k=4).
```

Foi criada uma função `kinduction_safety` que verifica a segurança da propriedade.

```
In [163._ def kinduction_safety(vars, init, trans, error, k, a, b, nbits):
with Solver(name="z3") as solver:
    s = [genState(vars, 'X', i, nbits) for i in range(k)]

    solver.add_assertion(init(s[0], a, b, nbits))

    for i in range(k-1):
        solver.add_assertion(trans(s[i], s[i+1], nbits))

    for i in range(k):
        solver.push()
        solver.add_assertion(error(s[i], nbits))
        if solver.solve():
            print(f"> Propriedade de segurança violada nos primeiros {k} estados!")
            print(f"> Erro encontrado no passo {i}")
            return False
        solver.pop()

    s2 = [genState(vars, 'Y', i + k, nbits) for i in range(k + 1)]

    for i in range(k):
        solver.add_assertion(Not(error(s2[i], nbits)))
        if i < k-1:
            solver.add_assertion(trans(s2[i], s2[i+1], nbits))

    solver.add_assertion(error(s2[-1], nbits))

    if solver.solve():
        print(f"> Propriedade de segurança violada no passo indutivo!")
        return False

    print(f"> Programa seguro! Provado por k-indução. (k={k})")
    return True
```

```
In [164._ vars = ['x', 'y', 'z', 'pc']
kinduction_safety(vars, init_c, trans, error, k, a_c, b_c, nbits)
```

```
> Programa seguro! Provado por k-indução. (k=4)
```

```
Out[164._ True
```